Java Programming: Multithreading and Concurrency

Study notes for a document-based Retrieval-Augmented Generation (RAG) project

How to use these notes

These notes provide connected explanations, examples, terminology, and review questions. They are intended as a starter corpus for testing PDF extraction, chunking, embedding, and retrieval. Read the sections and ask questions whose answers appear in the material.

1. Process and thread

A process is an executing program with its own resources and memory space. A thread is an execution path within a process. Multiple threads in one process share heap objects, which can make communication efficient but also creates the possibility of race conditions.

2. Creating threads

A task can be represented by implementing Runnable and passed to a Thread. Another approach is to use an ExecutorService, which manages a pool of worker threads. Separating a task from the thread that executes it makes code easier to reuse and test.

Code: Runnable

```
class PrintTask implements Runnable {
public void run() {
System.out.println("Task is running");
}
}

class Main {
public static void main(String[] args) {
Thread worker = new Thread(new PrintTask());
worker.start();
}
}
```

3. start and run

Calling start asks the JVM to schedule a new thread of execution; that thread invokes run. Calling run directly is an ordinary method call on the current thread and does not start a new thread. A Thread object cannot be started more than once.

4. Thread states

The Thread.State enumeration includes NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. These states describe broad lifecycle conditions. A RUNNABLE thread may be running or ready to run, depending on scheduling. Thread state is useful for diagnostics but should not be used as a synchronization mechanism.

5. Race conditions

A race condition occurs when the result depends on the unpredictable timing of concurrent operations. For example, two threads incrementing the same shared counter can lose updates if the read-modify-write sequence is not protected. Correct synchronization or thread-safe designs are needed for shared mutable state.

6. synchronized

The synchronized keyword provides mutual exclusion using an object's monitor. Only one thread at a time can hold a particular monitor. Synchronized blocks can protect a small critical section, while synchronized methods lock the associated object or class monitor depending on whether the method is an instance or static method.

Code: synchronized counter

```
class Counter {
private int value = 0;

public synchronized void increment() {
value++;
}

public synchronized int getValue() {
return value;
}
}
```

7. wait, notify, and notifyAll

Object.wait causes the current thread to release the object's monitor and wait until notified, interrupted, or timed out. The thread must own the object's monitor before calling wait. notify wakes one waiting thread; notifyAll wakes all waiting threads. Calls should be made while holding the same object's monitor, and wait should normally be used in a loop that checks the condition.

8. volatile

The volatile keyword provides visibility and ordering guarantees for reads and writes of a variable. A write to a volatile field happens-before subsequent reads of that field. Volatile does not make compound operations such as count++ atomic. Use atomic classes or locks when an operation must be indivisible.

9. ExecutorService

ExecutorService separates task submission from thread management. A fixed thread pool reuses a bounded number of worker threads. Submit can return a Future representing a pending result. Applications should shut down an executor when it is no longer needed so its worker threads can terminate.

10. Deadlock and starvation

Deadlock occurs when threads wait indefinitely for resources held by one another. Consistent lock ordering, small critical sections, and avoiding unnecessary nested locks can reduce risk. Starvation occurs when a thread is repeatedly denied access to resources or CPU time. Fairness policies may help in some situations but can reduce throughput.

11. Interruption

Interruption is a cooperative cancellation signal. A thread can be interrupted by another thread. Blocking methods such as sleep and wait may respond by throwing InterruptedException. Code should normally restore the interrupt status or propagate the exception when it cannot handle cancellation itself.

12. Review questions

1. What is the difference between start() and run()?

2. What problem does synchronized solve?

3. Why must wait() be called while owning the monitor?

4. Does volatile make count++ atomic?

5. What is deadlock?

End of document — Java study corpus